공연 좌석 예매 시스템 요구사항 명세서

항목	내용
문서 버전	v0.12 (초안)
작성일	2026-10-01
상태	작성 중
기술 스택	백엔드: Java 21, Spring Boot 4.1.0 (Spring Framework 7) DB: MySQL 8 (Amazon RDS) 프론트엔드: Vue.js 3 결제: 토스페이먼츠 결제위젯 v2 인프라: AWS EC2 1대에 Caddy(리버스 프록시·정적 파일·HTTPS)로 프론트엔드와 백엔드를 함께 배포 부하 테스트: k6 Redis: **보류** (도입 여부는 측정 결과로 결정, 7.3)
팀 구성	서창훈 (PM·인프라), 조은혜 (백엔드), 김민수 (프론트엔드) — 상세는 12장

변경 이력

버전	날짜	내용
v0.1	2026-10-01	최초 작성
v0.2	2026-10-02	단계 구분 삭제, 전체 범위를 한 번에 진행하도록 변경
v0.3	2026-10-02	Mock 결제를 토스페이먼츠 연동으로 교체(주문·승인·취소·웹훅·보정). 문서 내 모순 해소(FR-504, CS-05, CS-08, NFR-01과 대기열). 좌석 상태 `PAYING` 추가. 누락 정의 보완(선점 한도 범위, 입장 토큰, Redis 장애 동작, KOPIS 변경 반영, 부하 테스트 환경, 오류 코드, 데이터 모델). 결정 기록(7.3) 신설
v0.4	2026-10-02	미결 사항 확정: 대기열 활성 사용자 수 N 산정식(FR-403, FR-407), 예약 취소 기한(FR-606), KOPIS 출처 표기(FR-109)와 호출량 제어(FR-110), 부하 테스트 환경(10.2). 결정 근거를 7.3에 추가
v0.5	2026-10-02	기술 스택 확정(Spring Boot 4.1.0, MySQL 8, Vue.js 3, AWS). Redis 보류에 따라 대기열을 MySQL로 구현하고 Redis 의존 항목(전략 D·E, NFR-07, NFR-05 목표)을 보류 처리. 프론트엔드 요구사항(FR-8xx)과 AWS 구성(3.1) 추가. 부하 테스트 환경을 AWS로 변경. 3인 협업 방식(12장) 추가
v0.6	2026-10-02	역할 확정(서창훈 PM·인프라, 조은혜 백엔드, 김민수 프론트엔드). 백엔드 작업량 분산을 위해 KOPIS 동기화·좌석 데이터 생성·부하 테스트를 인프라 담당으로 이동. PM 업무(12.4)와 담당자별 작업 순서 추가
v0.7	2026-10-02	배포 구성을 EC2 1대 + Caddy로 변경(ECS·ALB·S3·CloudFront 제거). Docker Compose 배포, 같은 출처 구성으로 CORS 불필요. 부하 테스트 환경과 작업 순서를 이에 맞게 수정. 비밀값을 SSM Parameter Store로 변경
v0.8	2026-10-02	RDS 사용 확정. 도메인 준비 전 임시 운영 방식(Elastic IP + HTTP) 추가
v0.9	2026-10-02	관리자 기능과 관리자 화면을 범위에 포함(FR-9xx): 대시보드, 공연·회차 관리, 좌석 현황, 예약·결제 조회, 직권·회차 일괄 취소, 결제 이상 건 처리, 동기화 관리, 대기열 모니터링, 회원 관리, 감사 로그. 우선순위(P1/P2) 도입. 관련 데이터 모델·API·오류 코드·시나리오(CS-17, CS-18)·역할 분담 수정
v0.10	2026-10-02	협업 도구 확정: 이슈 관리 Jira, 소통 Slack. Jira·GitHub·Slack 연동 규칙과 채널 구성 추가(12.2, 12.5)
v0.11	2026-10-02	Jira 프로젝트 확정(예매한 사이, 키 KAN). 요구사항별 이슈 등록 완료
v0.12	2026-10-02	KOPIS 인증키 발급·확인 담당을 조은혜로 지정

표기: `신규` v0.3에서 추가된 항목, `변경` v0.3에서 내용이 바뀐 항목

---

1. 개요

1.1 목적

인기 공연의 예매 오픈 시점처럼 짧은 시간에 요청이 몰리는 상황에서도 같은 좌석이 두 번 팔리지 않고, 서버가 무너지지 않으며, 결제된 금액과 확정된 좌석이 항상 일치하는 예매 시스템을 설계·구현하고, 그 결과를 부하 테스트로 수치화하여 검증한다.

1.2 해결하려는 문제

- 여러 사용자가 동시에 같은 좌석을 선택하면 중복 예약(오버부킹)이 발생할 수 있다.
- 오픈 직후 트래픽이 한꺼번에 DB로 몰리면 응답 지연과 장애가 발생한다.

- 좌석을 선점한 뒤 결제하지 않고 이탈하면 좌석이 묶여 판매 기회가 사라진다.
- 네트워크 재시도로 같은 결제 요청이 여러 번 처리될 수 있다.
- 외부 결제(PG) 승인 결과와 내부 예약 상태가 어긋날 수 있다. 예: 돈은 빠져나갔는데 좌석은 확정되지 않음, 좌석은 확정됐는데 승인은 실패함. 신규

1.3 범위 변경

포함 (In scope)

- KOPIS 오픈API 기반 공연·공연시설 데이터 수집 및 저장
- 공연·회차·좌석 조회
- 예매 대기열
- 좌석 임시 선점(Hold) 및 자동 만료
- 토스페이먼츠 결제위젯 연동(테스트 키 환경): 결제 승인, 전체 취소(환불), 웹훅 수신, 결과 불명 시 보정
- 부하 테스트·장애 재현용 가짜 결제 게이트웨이(Fake)
- 동시성 제어 전략 구현 및 부하 테스트 비교 (Redis 기반 전략 D·E는 보류)
- Vue.js 사용자 웹 화면 (공연 조회, 대기열, 좌석 선택, 결제위젯, 예약 관리)
- 관리자 기능과 관리자 웹 화면 (운영 조회, 직권 취소·환불, 결제 이상 건 처리, 동기화·대기열 관리, 감사 로그)
- AWS 배포 (개발 환경, 부하 테스트 환경)

제외 (Out of scope)

- 토스페이먼츠 라이브 키 발급·전자결제 계약 및 실제 금전 거래
- 가상계좌·계좌이체 등 입금 대기가 있는 결제수단 (카드·간편결제만 사용)
- 부분 취소, 취소 수수료, 할인·쿠폰·포인트, 양도 정책
- 실제 공연장 좌석 배치도 재현 (규칙 기반으로 생성)
- KOPIS 공연 취소에 따른 자동 일괄 환불 (관리자가 실행하는 회차 단위 일괄 취소는 포함, FR-907)
- 운영(프로덕션) 환경 구축
- Redis 도입 (보류. 도입이 결정되면 7.1 전략 D·E, NFR-07을 범위에 다시 넣는다)
- 모바일 앱, 알림(SMS/푸시)

1.4 용어 정의

용어	정의
공연 (Performance)	KOPIS에서 수집한 공연 단위. KOPIS ID(`mt20id`)로 식별
공연시설 (Facility)	공연장. KOPIS ID(`mt10id`)로 식별
회차 (Schedule)	특정 공연의 특정 일시 1회 공연
좌석 (Seat)	회차별로 존재하는 판매 단위
선점 (Hold)	결제 전 일정 시간 동안 좌석을 다른 사용자가 고르지 못하게 잡아두는 상태
주문 (Order)	선점한 좌석으로 결제를 시작할 때 만드는 결제 대상. 토스페이먼츠의 `orderId` 와 1:1로 대응하며, 이 문서에서는 Reservation 엔티티가 주문 역할을 겸한다 신규
예약 (Reservation)	주문이 결제 승인되어 확정된 좌석 구매 (`CONFIRMED` 상태의 주문)
결제 기한	주문 생성 시각부터 10분. 토스페이먼츠 결제 인증 유효 시간(10분)에 맞춘다. 기한 안에 승인되지 않은 주문은 만료된다 신규
보상 취소	PG 승인은 성공했지만 내부 예약 확정에 실패했을 때, 승인을 취소하여 돈과 좌석 상태를 맞추는 처리 신규
보정 (Reconciliation)	승인 결과를 알 수 없을 때 PG 조회 API로 실제 결제 상태를 확인하여 내부 상태를 맞추는 처리 신규
대기열 (Queue)	오픈 시 예매 페이지 진입 순서를 관리하는 줄
입장 토큰	대기열을 통과한 사용자에게 발급되는, 선점·주문 생성 API 호출 자격. 서명된 JWT로 발급한다

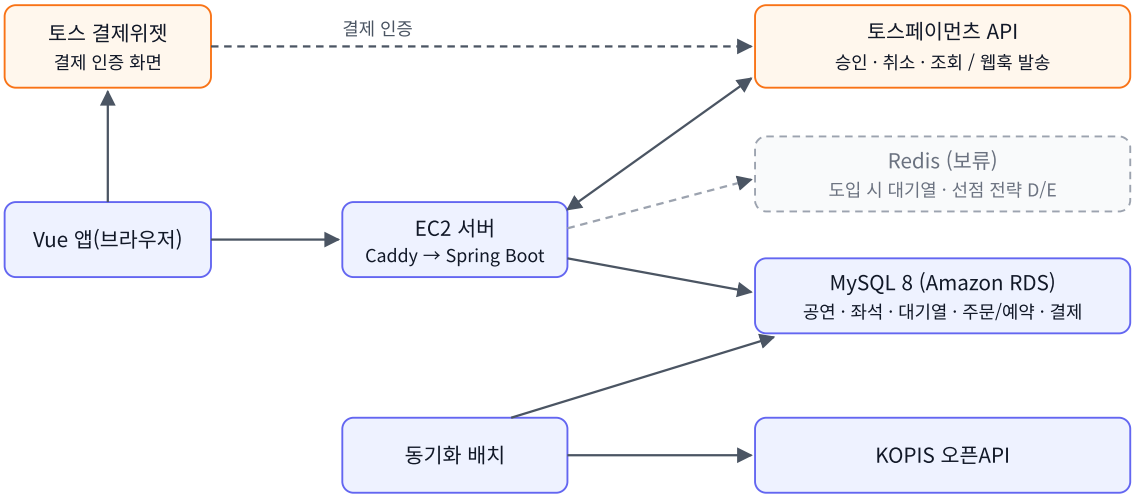
---

2. 사용자와 역할

역할	설명	주요 권한
비회원	로그인하지 않은 방문자	공연·회차·좌석 조회
회원	로그인한 사용자	대기열 진입, 좌석 선점, 주문·결제, 예약 조회·취소

관리자	운영자 (`Member.role = ADMIN`, 시드 데이터로 생성)	4.9의 관리자 기능 전체 (공연·회차 관리, 예약·결제 조회, 직권 취소·환불, 결제 이상 건 처리, 동기화·대기열·회원 관리)

3. 시스템 구성 개요 변경



핵심 원칙

- 예매 흐름(좌석 조회·선점·주문)은 KOPIS를 호출하지 않는다. KOPIS 데이터는 동기화 배치가 DB에 적재한 것만 사용한다.
- 결제 금액은 서버가 좌석 가격으로 계산하여 주문에 저장한 값만 신뢰한다. 클라이언트가 보낸 금액은 검증 대상일 뿐이다.
- 예약 확정 여부의 최종 기준은 토스페이먼츠의 결제 상태다. 내부 상태와 다르면 보정 또는 보상 취소로 맞춘다.
- 결제 게이트웨이는 인터페이스로 분리한다. 기본 구현은 토스페이먼츠, 부하 테스트·장애 재현용으로 Fake 구현을 둔다.

3.1 AWS 구성 변경

EC2 1대에서 Docker Compose로 Caddy와 Spring Boot 컨테이너를 실행한다. Caddy가 Vue 빌드 결과(정적 파일)를 직접 서빙하고 `/api/*` 요청만 Spring Boot로 넘긴다. 프론트와 API가 같은 도메인(같은 출처)이므로 CORS 설정이 필요 없고, Caddy가 Let's Encrypt 인증서를 자동으로 발급·갱신한다.

구성 요소	AWS 서비스 / 도구	비고
애플리케이션 서버	EC2 1대 + Elastic IP	Docker Compose: `caddy`, `app` (Spring Boot) 컨테이너
웹 서버	Caddy 2	HTTPS 자동 발급, Vue 정적 파일 서빙, `/api/*` → `app:8080` 리버스 프록시, SPA 라우팅은 `try_files {path} /index.html`
데이터베이스	Amazon RDS for MySQL 8	프라이빗 서브넷, EC2 보안 그룹에서만 접근. 단일 AZ
도메인	Route 53 (또는 외부 등록 도메인) — 추후 준비	HTTPS 인증서 발급과 토스 웹훅 수신 URL에 필요. 준비 전에는 아래 임시 운영 방식을 따른다
이미지 저장소	ECR	GitHub Actions가 `app` 이미지와 Vue 빌드를 포함한 `caddy` 이미지를 빌드·푸시
배포	GitHub Actions → SSH	EC2에서 `docker compose pull && docker compose up -d`. 단일 서버라 배포 중 수 초 중단을 허용(dev 환경)
비밀값	SSM Parameter Store (SecureString)	DB 비밀번호, 토스 시크릿 키, KOPIS 키, JWT 서명 키. 배포 시 EC2 IAM 역할로 읽어 환경변수로 주입 (NFR-09)
보안 그룹	EC2: 80·443 공개, 22는 팀원 IP만. 8080은 외부 미노출	RDS: 3306을 EC2 보안 그룹에만 허용
로그·알람	CloudWatch Logs / Alarms	보상 취소 실패, 보정 지연, EC2 CPU·메모리 알람. 알람은 Slack `#alerts` 채널로 보낸다(12.5)
메트릭	Prometheus + Grafana (별도 EC2)	부하 테스트 환경에서만 기동하여 측정 대상 서버에 부하를 주지 않음 (NFR-10)

인프라 코드	Terraform	EC2·RDS·보안 그룹·IAM을 코드로 관리하고 부하 테스트 환경은 테스트 후 삭제

> **도메인 준비 전 임시 운영**
>
> - Caddy는 Elastic IP로 HTTP(80)만 서빙한다(`:80` 사이트 블록). Spring Boot 설정과 Vue 코드에는 주소를 하드코딩하지 않고 환경변수(`APP_BASE_URL`)로 받아, 도메인이 생기면 Caddyfile과 환경변수만 바꾼다.
> - 토스 결제위젯의 `successUrl` · `failUrl` 은 `APP_BASE_URL` 기준으로 만든다. 결제 승인·실패·취소 흐름은 IP 주소 환경에서 먼저 확인한다.
> - 토스 웹훅 수신(FR-611)과 HTTPS 동작은 도메인 준비 후 dev 환경에서 확인한다. 그 전에는 Fake 게이트웨이 테스트(CS-16)와 로컬 터널링으로 검증한다.
> - HTTP 구간에서는 로그인 토큰이 암호화되지 않으므로 dev 환경에 실제 개인정보·실제 비밀번호를 쓰지 않는다.

환경	구성	결제
local	개인 PC, Docker Compose로 MySQL 실행. Vue는 Vite 개발 서버의 프록시로 `/api` 를 로컬 백엔드에 연결	Fake 게이트웨이 또는 토스 테스트 키
dev	EC2 t3.medium(2 vCPU / 4 GB) 1대 + RDS db.t3.micro, 상시 운영, `main` 브랜치 자동 배포. 토스 웹훅 수신 URL로 사용	토스 테스트 키
loadtest	dev와 같은 구성(EC2 1대 + Caddy + RDS)을 성능 고정형 인스턴스로 생성(10.2). 테스트할 때만 Terraform으로 생성·삭제	Fake 게이트웨이

4. 기능 요구사항 (기능 명세)

각 기능은 ID, 요구사항, 인수 기준으로 정의한다. 이 장이 프로젝트의 기능 명세서 역할을 하며, API 명세(OpenAPI)·화면 설계·DB 설계는 이 장의 ID를 참조해 작성한다.

4.1 외부 데이터 연동 (FR-1xx)

ID	요구사항	인수 기준
FR-101	시스템은 매일 정해진 시각(04:00 KST)에 KOPIS 공연 목록을 동기화한다 변경	수집 대상 기간은 기준일 −30일 ~ +180일(설정값). 기간을 31일 단위로 나누어 요청하고, 100건 단위로 페이징하여 끝까지 수집한다
FR-102	신규 공연은 공연 상세와 공연시설 상세를 추가 조회하여 저장한다 변경	이미 저장된 공연은 목록 응답의 상태·기간이 바뀐 경우에만 상세를 재조회한다. 추가로 공연 예정·공연중 공연은 7일마다 상세를 재조회하여 가격·시간 정보를 갱신한다
FR-103	공연시설 상세의 좌석 수를 저장한다	같은 시설은 한 번의 동기화 실행에서 한 번만 조회한다(시설 단위 캐시)
FR-104	가격 안내 문자열을 파싱하여 좌석 등급과 가격으로 변환한다 변경	"R석 66,000원, S석 44,000원" → R:66000, S:44000. "전석 30,000원" → 단일 등급 "전석". "무료" 또는 0원 → 판매 상태 `NOT_FOR_SALE` (PG 결제 불가). 파싱 실패 시 단일 등급 "일반석", 기본 가격 50,000원(설정값)으로 저장하고 경고 로그를 남긴다
FR-105	빈 문자열·누락 값은 null로 정규화하여 저장한다	필수 필드(공연 ID, 공연명, 기간) 누락 건은 저장하지 않고 실패 이력(SyncFailure)에 기록한다
FR-106	동기화 실패 시 재시도한다	요청 단위로 최대 3회, 지수 백오프(1s, 2s, 4s). 최종 실패 건은 SyncFailure에 요청 파라미터·오류 내용과 함께 기록
FR-107	관리자는 전체 재동기화를 수동 실행할 수 있다	실행 결과(수집·신규·갱신·실패 건수, 소요 시간)를 응답한다. 동기화가 이미 실행 중이면 409 `SYNC_IN_PROGRESS`
FR-108	KOPIS에서 공연 상태가 취소되었거나 수집 대상에서 사라진 공연은 판매를 중지한다 신규	판매 상태를 `SALE_STOPPED` 로 바꾸고 신규 선점·주문을 거절한다. 기존 예약이 있으면 관리자 대시보드의 확인 필요 항목에 표시하고(FR-902), 관리자가 회차 단위 일괄 취소(FR-907)로 처리한다. 자동 환불은 하지 않는다
FR-109	KOPIS 데이터의 출처를 표기한다 신규	공연 목록·상세 API 응답에 `source` 필드("자료 출처: 공연예술통합전산망(KOPIS), www.kopis.or.kr")를 포함하고, 화면 하단과 README에 같은 문구를 표기한다. 공연 포스터는 KOPIS가 제공한 URL을 그대로 링크하며 별도로 복제·가공하지 않는다
FR-110	KOPIS 호출량을 제어하고 기록한다 신규	호출 속도는 초당 최대 5회(설정값). 동기화 실행마다 호출 수를 SyncHistory에 기록하고 당일 누적 호출 수를 집계한다. 일일 호출 예산(설정값, 기본 1,000회)의 80%에 도달하면 경고 로그를 남기고, 100%에 도달하면 남은 상세 조회를 다음 실행으로 미룬다(목록 수집은 우선 완료)

4.2 공연·회차·좌석 데이터 생성 (FR-2xx)

KOPIS는 좌석 배치와 회차별 판매 정보를 제공하지 않으므로, 아래 규칙으로 예매용 데이터를 생성한다.

ID	요구사항	인수 기준
FR-201	관리자는 공연에 회차(일시)를 등록할 수 있다 변경	공연 기간 밖의 일시, 같은 공연의 같은 일시 중복은 등록할 수 없다. 선점·예약이 하나라도 있는 회차는 일시를 바꾸거나 삭제할 수 없다
FR-202	회차가 생성되면 시설 좌석 수만큼 좌석을 자동 생성한다 변경	한 열 20석 기준으로 행을 나누고(A열, B열…), 마지막 행에는 나머지 좌석만 둔다. 좌석 수가 없으면 기본 200석
FR-203	좌석 등급은 앞 열부터 가격이 높은 등급 순으로 배정한다 변경	등급이 n개면 전체 행을 n등분하고, 나누어떨어지지 않는 나머지 행은 가장 낮은 등급에 배정한다. 등급 수가 행 수보다 많으면 가격이 높은 등급부터 한 행씩 배정하고 남는 등급은 사용하지 않으며 경고 로그를 남긴다
FR-204	관리자는 회차별 예매 오픈 시각을 설정할 수 있다 변경	오픈 시각은 회차 시작 시각보다 앞서야 한다. 오픈 전에는 대기열 진입·좌석 선점 요청이 거절된다(403 `SALE_NOT_OPEN`). 미설정 시 기본값은 공연 시작일 7일 전 14:00(설정값)
FR-205	공연 시간 안내 텍스트로부터 회차를 자동 생성한다 변경	"화요일 ~ 금요일(19:30), 토요일 ~ 일요일(14:00,18:00)"처럼 요일(범위)과 시각 목록으로 된 형식만 지원하고, 공연 기간 안의 날짜에 대해 생성한다. "HOL" 등 지원하지 않는 토큰은 무시한다. 하나도 생성하지 못한 공연은 판매 상태 `NO_SCHEDULE`("회차 미등록")로 표시

4.3 조회 (FR-3xx)

ID	요구사항	인수 기준
FR-301	사용자는 공연 목록을 조회할 수 있다	장르·지역·상태(공연예정/공연중) 필터, 페이징 지원
FR-302	사용자는 공연 상세와 회차 목록을 조회할 수 있다	회차별 잔여 좌석 수(`AVAILABLE` 좌석 수)가 표시된다. 최대 1초 지연을 허용한다
FR-303	사용자는 회차의 좌석 배치와 좌석 상태를 조회할 수 있다 변경	`HELD` · `PAYING` · `RESERVED` 좌석은 선택 불가로 표시된다. 선점 만료 시각이 지난 좌석은 스케줄러 반영 전이라도 선택 가능으로 표시한다(지연 판정). 비회원도 조회할 수 있으며 최대 1초 지연을 허용하고, 최종 판정은 선점 요청 시점에 한다

4.4 대기열 (FR-4xx)

Redis 보류에 따라 대기열은 MySQL로 구현한다(`QueueEntry` 테이블). 자동 증가 ID가 순번이 되고, `(scheduleId, memberId)` 유니크 제약으로 중복 진입을 막는다(CS-10). 입장 토큰 발급 스케줄러는 1초 주기로 동작하며, 여러 API 서버 중 한 곳에서만 실행되도록 ShedLock(JDBC)으로 잠근다. 대기열은 인터페이스로 분리하여 Redis 도입 시 구현만 바꾼다.

ID	요구사항	인수 기준
FR-401	예매 오픈 시각 이후 회원은 회차별 대기열에 진입한다 변경	진입 순서대로 순번이 부여된다. 오픈 전 진입은 403 `SALE_NOT_OPEN`
FR-402	회원은 자신의 현재 순번과 예상 대기 시간을 조회할 수 있다 변경	폴링 주기 3초 이상. 예상 대기 시간 = 내 앞 인원 ÷ 최근 1분간 평균 입장 처리 속도(명/초)
FR-403	동시 활성 사용자가 N명 미만이면 다음 순번에게 입장 토큰을 발급한다 변경	N은 7.3의 산정식으로 정하며 10초마다 다시 계산한다(측정 전 초기값 100). 활성 슬롯은 ① 입장 토큰 만료, ② 해당 회차 예약 확정, ③ 대기열 이탈 요청 중 먼저 일어난 시점에 반납된다
FR-404	입장 토큰은 발급 후 10분이 지나면 만료된다 변경	토큰은 회원 ID·회차 ID·만료 시각을 담은 서명 JWT이며 Redis 없이 서명만으로 검증한다. 만료된 토큰으로는 좌석 선점·주문 생성 API를 호출할 수 없다(403 `QUEUE_TOKEN_REQUIRED`). 이미 생성된 주문의 결제 승인은 토큰이 아닌 결제 기한으로 제어한다
FR-405	같은 회원이 중복 진입하면 기존 순번을 유지한다	새로고침해도 순번이 뒤로 밀리지 않는다. 이미 입장 토큰을 받은 회원이 다시 진입하면 기존 토큰을 돌려준다
FR-406	회원은 대기열에서 이탈할 수 있다 신규	이탈하면 순번 또는 활성 슬롯이 즉시 제거된다
FR-407	판매 가능한 좌석이 없으면 입장 토큰 발급을 멈춘다 신규	`AVAILABLE` 좌석이 0이면 신규 토큰을 발급하지 않고 순번 조회 응답에 `soldOut: true` 를 담는다. 선점 만료·결제 실패·취소로 좌석이 돌아오면 발급을 재개한다

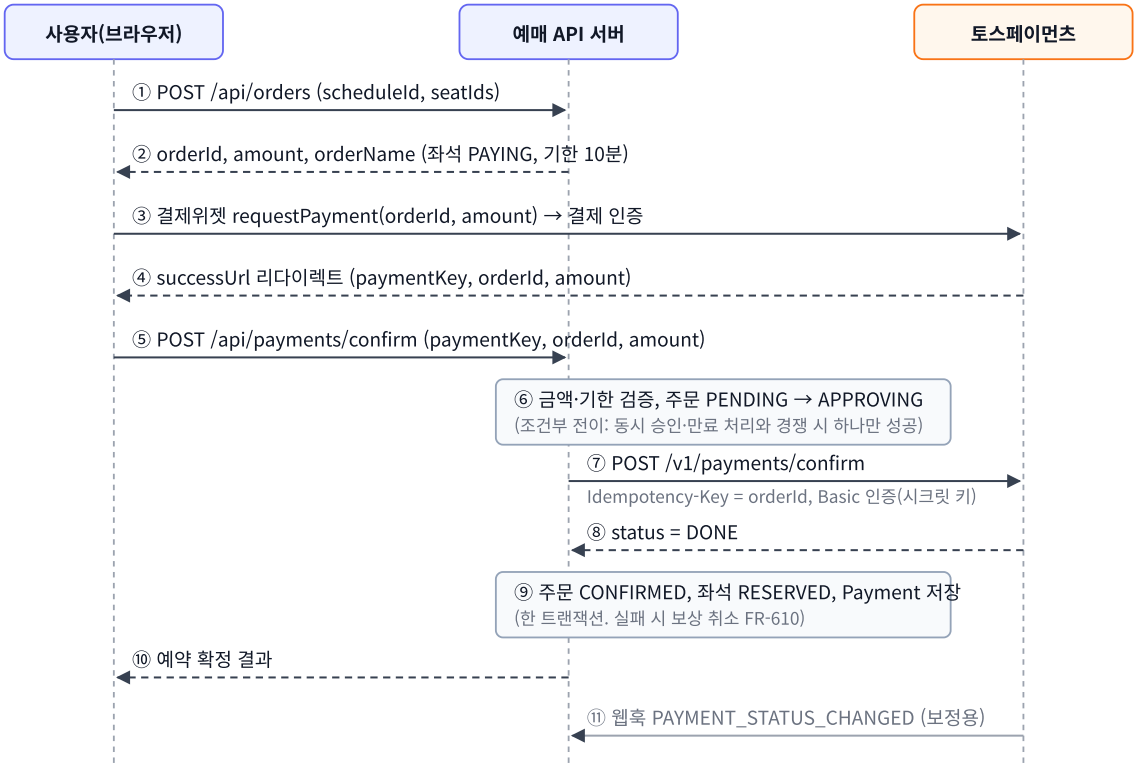
4.5 좌석 선점 (FR-5xx)

ID	요구사항	인수 기준
FR-501	회원은 유효한 입장 토큰으로 좌석을 선택하여 선점할 수 있다	이미 선점·결제 중·예약된 좌석이면 409 `SEAT_ALREADY_TAKEN`. 판매 상태가 `ON_SALE`가 아닌 공연이면 403 `SALE_NOT_OPEN`

FR-502	선점은 5분간 유지된다 변경	5분이 지나면 자동으로 `AVAILABLE` 로 돌아간다(NFR-08). 주문을 생성하면(FR-607) 좌석은 `PAYING` 이 되어 선점 만료 대상에서 빠지고, 이후에는 결제 기한(10분)이 적용된다
FR-503	회원은 회차당 최대 4석까지 보유할 수 있다 변경	한도는 해당 회원의 같은 회차 `HELD` + `PAYING` + `RESERVED` 좌석 수의 합으로 계산한다. 초과 요청은 409 `HOLD_LIMIT_EXCEEDED`
FR-504	여러 좌석을 한 번에 선점하면 전부 성공하거나 전부 실패한다 확정	일부만 선점된 상태가 남지 않는다. 실패 응답에는 선점할 수 없었던 좌석 ID 목록을 담는다
FR-505	회원은 선점을 직접 해제할 수 있다	해제된 좌석은 즉시 다른 사용자가 선택 가능. `PAYING` 상태 좌석은 해제할 수 없다(409 `INVALID_STATE_TRANSITION`)

4.6 주문·결제·예약 (FR-6xx) 변경

토스페이먼츠 결제위젯 v2를 사용한다. 흐름은 **주문 생성 → 결제 인증(토스 화면) → 승인 → 예약 확정**이다. v0.2와의 추적성을 위해 기존 ID(FR-601 ~ FR-606)는 유지하고 새 요구사항은 FR-607부터 붙였으므로, 흐름 순서는 FR-607(주문 생성) → FR-601(승인·확정) → FR-608 ~ FR-611(실패·보정·보상 취소·웹훅)이다.



ID	요구사항	인수 기준
FR-601	회원은 결제 인증 후 결제 승인을 요청하여 예약을 확정할 수 있다 변경	서버는 요청 금액이 주문 금액과 같은지 확인하고(다르면 400 `AMOUNT_MISMATCH` , 토스 호출 없음), 주문을 `PENDING → APPROVING` 으로 조건부 전이한 뒤 토스 승인 API를 호출한다. 승인 응답의 금액도 주문 금액과 대조한다. 성공 시 한 트랜잭션에서 주문 `CONFIRMED` , 좌석 `RESERVED` , Payment 저장
FR-602	결제 기한이 지난 주문은 승인하지 않는다 변경	기한 후 승인 요청은 토스를 호출하지 않고 410 `ORDER_EXPIRED` . 만료 스케줄러는 `PENDING` 주문만 `EXPIRED` 로 바꾸고 좌석을 `AVAILABLE` 로 돌린다. `APPROVING` 주문은 만료 처리하지 않는다
FR-603	결제 승인 요청은 멱등하게 처리된다 변경	멱등 기준은 `orderId` 다. 이미 `CONFIRMED` · `FAILED` 인 주문의 재요청에는 저장된 최초 결과를 돌려준다. 처리 중(`APPROVING`)이면 409 `PAYMENT_IN_PROGRESS` . 토스 승인 호출에는 `Idempotency-Key: {orderId}` 를 붙인다(토스 보관 15일)
FR-604	결제 게이트웨이를 설정으로 교체할 수 있다 신규	기본은 토스페이먼츠 구현. `loadtest` · `test` 프로필에서는 Fake 구현을 쓰며, 성공·실패·지연·타임아웃·결과 불명(승인됐지만 응답 유실)을 설정으로 재현할 수 있다
FR-605	회원은 자신의 예약 목록을 조회할 수 있다	`CONFIRMED` · `CANCELED` 주문을 최신순으로 보여준다

FR-606	회원은 회차 시작 24시간 전까지 예약을 취소할 수 있다 변경	취소 기한은 설정값(기본 24시간)이며, 기한이 지나면 403 `CANCEL_PERIOD_EXPIRED`. 예약 전체 취소만 지원하고 수수료는 없다. 주문을 `CONFIRMED → CANCELING` 으로 조건부 전이한 뒤 토스 취소 API(`Idempotency-Key: cancel-{orderId}`)를 호출한다. 성공 시 한 트랜잭션에서 주문 `CANCELED`, 좌석 `AVAILABLE`. 실패 시 `CONFIRMED` 로 되돌리고 502 `PAYMENT_GATEWAY_ERROR`. 환불 완료 전에는 좌석이 다른 사용자에게 열리지 않는다
FR-607	회원은 자신이 선점한 좌석으로 주문을 생성할 수 있다 신규	유효한 입장 토큰이 필요하다. 요청 좌석이 모두 본인 소유의 만료되지 않은 선점이어야 하며 아니면 410 `HOLD_EXPIRED` 또는 403 `FORBIDDEN_RESOURCE`. 서버가 좌석 등급 가격으로 금액을 계산해 주문(`PENDING`)과 좌석별 가격 스냅샷을 저장하고, `orderId` (영문·숫자· `-` · `_` , 6~64자, 전역 유일)를 발급한다. 좌석은 `HELD → PAYING`, 결제 기한 = 생성 시각 + 10분
FR-608	결제가 실패하면 주문을 실패 처리하고 좌석을 되돌린다 신규	failUrl 리다이렉트 또는 승인 거절 시 주문 `FAILED`. 좌석은 원래 선점 만료 시각 전이면 `HELD` 로 돌아가 재시도할 수 있고(새 주문 생성), 지났으면 `AVAILABLE`. 재시도 시에도 FR-503 한도와 입장 토큰 유효성은 다시 검사한다
FR-609	승인 결과를 알 수 없으면 보정한다 신규	승인 API 타임아웃(연결 5초·읽기 30초)·5xx 시 주문을 `APPROVING` 으로 두고 즉시 1회, 이후 1분 주기 스케줄러로 `GET /v1/payments/orders/{orderId}` 를 조회한다. `DONE` 이면 확정, 승인되지 않았으면 FR-608과 같이 실패 처리. 2분 넘게 `APPROVING` 인 주문은 모두 보정 대상이다
FR-610	승인 성공 후 내부 반영에 실패하면 보상 취소한다 신규	토스 승인은 성공했으나 예약 확정 트랜잭션이 실패하면 토스 취소 API를 `Idempotency-Key: cancel-{orderId}` 로 호출하고 주문을 `FAILED` 로 둔다. 취소 호출이 실패하면 지수 백오프로 재시도하고 실패 로그·메트릭을 남긴다. 결제됐는데 좌석이 확정되지 않은 상태가 남지 않아야 한다
FR-611	토스 웹훅을 수신하여 상태를 맞춘다 신규	`PAYMENT_STATUS_CHANGED` 를 수신하면 10초 안에 200을 응답하고 비동기로 처리한다. 웹훅 본문은 신뢰하지 않고 결제 조회 API로 다시 확인한 뒤 반영한다. 같은 웹훅이 여러 번 와도 결과가 같다. 토스 상점관리자에서 취소된 결제는 예약 `CANCELED`, 좌석 `AVAILABLE` 로 반영한다
FR-612	결제수단은 카드·간편결제로 한정한다 신규	결제위젯 설정에서 가상계좌·계좌이체를 노출하지 않는다. 승인 응답이 `WAITING_FOR_DEPOSIT` 이면 보상 취소한다

4.7 회원 (FR-7xx)

ID	요구사항	인수 기준
FR-701	사용자는 회원가입·로그인할 수 있다	JWT 기반 인증. 액세스 토큰 유효 시간 1시간(리프레시 토큰은 범위 제외). 회원마다 추측할 수 없는 `customerKey` (UUID)를 발급해 결제위젯에 사용한다
FR-702	관리자 API는 관리자 역할만 호출할 수 있다 신규	일반 회원 호출 시 403 `FORBIDDEN_RESOURCE`. 관리자 계정은 시드 데이터로 생성한다. 관리자 화면(`/admin` 경로)은 라우터 가드로 관리자만 진입할 수 있고, 권한 판단은 항상 서버에서 한다

회원 기능은 동시성 검증의 대상이 아니므로 최소 구현으로 한정한다.

4.8 프론트엔드 (FR-8xx) 신규

Vue 3 + Vite, Vue Router, Pinia, Axios를 사용한다. 토스 결제위젯은 npm 패키지 `@tosspayments/tosspayments-sdk` (v2)로 연동하며 위젯용 클라이언트 키(`test_gck_`)를 사용한다. API 오류는 9.1의 오류 코드를 기준으로 공통 처리하고, 401이면 로그인 화면으로 보낸다.

ID	요구사항	인수 기준
FR-801	공연 목록·상세 화면	장르·지역·상태 필터와 페이징, 회차별 잔여석과 판매 상태(오픈 예정·판매 중·매진·판매 중지)를 표시한다. 화면 하단에 KOPIS 출처를 표기한다(FR-109)
FR-802	대기열 화면	3초 간격으로 순번과 예상 대기 시간을 갱신한다. 입장 토큰을 받으면 좌석 선택 화면으로 이동하고, `soldOut` 이면 매진 안내를 보여준다. "대기 취소" 버튼으로 이탈한다(FR-406)
FR-803	좌석 선택 화면	좌석 배치를 등급별 색으로 표시하고 선택 불가 좌석을 구분한다. 최대 4석까지만 선택된다. 선점 실패(409) 시 실패한 좌석을 표시하고 좌석 상태를 다시 불러온다. 선점 남은 시간을 타이머로 보여주고 해제할 수 있다
FR-804	결제 화면 (토스 결제위젯)	주문 생성 응답의 금액으로 `setAmount` 를 호출한다(클라이언트에서 금액을 계산하지 않음). `renderPaymentMethods` · `renderAgreement` 로 위젯을 그리고, `requestPayment` 에 `orderId`, `orderName`, `successUrl` (`/payments/success`), `failUrl` (`/payments/fail`)을 넘긴다. 결제 기한 타이머를 보여주고 결제 버튼 중복 클릭을 막는다

FR-805	결제 성공 처리 화면	쿼리의 `paymentKey`, `orderId`, `amount` 로 승인 API를 호출한다. 200이면 예약 완료, 202 또는 409 `PAYMENT_IN_PROGRESS` 이면 주문 상태를 2초 간격으로 최대 2분 폴링한다. 새로고침으로 다시 호출되어도 서버 멱등성(FR-603)으로 결과가 같다
FR-806	결제 실패 처리 화면	쿼리의 `code`, `message`, `orderId` 로 실패 통지 API를 호출하고 사유를 보여준다. 좌석이 `HELD` 로 돌아왔으면 "다시 결제"(새 주문 생성), 아니면 좌석 선택으로 안내한다
FR-807	로그인·회원가입·내 예약 화면	액세스 토큰은 `sessionStorage` 에 저장한다(리프레시 토큰 범위 제외에 따른 단순화). 내 예약 목록에서 취소할 수 있고, 취소 기한(FR-606)이 지난 예약은 취소 버튼을 비활성화한다

4.9 관리자 (FR-9xx) 신규

관리자 화면은 사용자 앱과 같은 Vue 프로젝트의 `/admin` 경로에 두고, 표·폼이 많으므로 컴포넌트 라이브러리 Element Plus를 사용한다(관리자 화면에만 적용, 지연 로딩). 관리자 API는 `/api/admin/**` 아래에 둔다. 우선순위 **P1**은 운영에 반드시 필요한 기능, **P2**는 일정이 밀리면 뒤로 미룰 수 있는 기능이다.

ID	순위	요구사항	인수 기준
FR-901	P1	관리자 레이아웃과 접근 제어	관리자로 로그인하면 관리자 메뉴가 보인다. 관리자 화면의 모든 API는 서버에서 `ADMIN` 역할을 검사한다(FR-702)
FR-902	P2	대시보드	오늘·내일 오픈하는 회차, 회차별 판매율(예약 좌석 ÷ 전체 좌석), 결제 이상 건 수(FR-908), 최근 동기화 결과, 확인 필요 항목(판매 중지 공연에 남은 예약, FR-108)을 한 화면에 보여준다. 1분마다 새로 고친다
FR-903	P1	공연 관리	공연을 이름·장르·지역·판매 상태로 검색한다. 판매 상태를 수동으로 `ON_SALE` ↔ `SALE_STOPPED` 전환할 수 있다(중지 사유 필수). 가격 등급은 선점·예약이 하나도 없을 때만 수정할 수 있다
FR-904	P1	회차 관리	공연별 회차 목록에서 회차 등록(FR-201), 오픈 시각 설정(FR-204), 일시 수정·삭제(선점·예약이 없을 때만, 있으면 409 `RESOURCE_IN_USE`)를 한다. 자동 생성된 회차(FR-205)는 자동 생성 표시를 붙인다
FR-905	P1	회차 좌석 현황	회차의 좌석 배치를 상태별 색으로 보여주고 상태별 좌석 수를 집계한다. 좌석을 누르면 선점자·주문·예약 정보를 볼 수 있다. 5초마다 새로 고친다
FR-906	P1	예약·결제 조회	회차, 회원 이메일, `orderId`, 주문 상태, 기간으로 검색하고 페이징한다. 상세 화면에서 좌석·금액·결제 정보(paymentKey, 결제수단, 승인·취소 시각)와 결제 통신 이력(PaymentEvent)을 시간순으로 보여준다
FR-907	P1	직권 취소·환불	관리자는 취소 기한(FR-606)과 관계없이 예약을 전체 취소할 수 있다(사유 필수). 처리 방식은 회원 취소와 같다(`CONFIRMED → CANCELING → CANCELED`, 토스 취소). **회차 일괄 취소**: 회차를 판매 중지한 뒤 그 회차의 모든 `CONFIRMED` 예약을 백그라운드 작업으로 취소한다. 진행률(전체·성공·실패)을 보여주고, 실패 건만 다시 실행할 수 있다. 같은 회차에 일괄 취소 작업이 이미 실행 중이면 409 `BULK_CANCEL_IN_PROGRESS`
FR-908	P1	결제 이상 건 처리	① 2분 넘게 `APPROVING` 인 주문, ② 보상 취소가 실패한 주문, ③ 웹훅·조회 결과와 내부 상태가 다른 주문을 목록으로 보여준다. 건별로 "토스 상태 다시 조회"(FR-609 보정 즉시 실행)와 "보상 취소 재시도"(FR-610)를 실행할 수 있다. 관리자가 상태를 직접 덮어쓰는 기능은 두지 않는다
FR-909	P1	동기화 관리	수동 동기화 실행(FR-107), 동기화 이력(SyncHistory) 목록, 실행별 실패 건(SyncFailure) 상세, 실패 건 재시도를 제공한다. 당일 KOPIS 호출 수와 예산 대비 사용률(FR-110)을 보여준다
FR-910	P2	대기열 모니터링	회차별 대기 인원, 활성 인원, 현재 N 값, 최근 1분 입장 속도를 5초마다 보여준다. 관리자는 회차별 N 상한을 수동으로 지정할 수 있고, 지정하면 산정식(7.3) 결과와 지정값 중 작은 값을 쓴다
FR-911	P2	회원 관리	이메일로 회원을 검색하고 예약 이력을 본다. 회원을 정지·해제할 수 있다(사유 필수). 정지된 회원은 로그인·대기열 진입·선점이 거절되며(403 `MEMBER_SUSPENDED`), 이미 확정된 예약은 유지된다
FR-912	P1	감사 로그	관리자의 모든 변경 작업(판매 상태 변경, 회차 수정·삭제, 직권·일괄 취소, 보정·재시도 실행, 동기화 실행, N 지정, 회원 정지)을 누가·언제·무엇을·왜(사유)·변경 전후 값과 함께 기록한다. 기록은 수정·삭제할 수 없다. 조회 화면(필터: 관리자, 작업 종류, 기간)은 P2

5. 비기능 요구사항

목표 수치는 초기 가설이며, 부하 테스트 결과에 따라 갱신한다. 성능 수치는 10.2의 테스트 환경을 기준으로 한다.

ID	분류	요구사항	검증 방법
NFR-01	정합성	좌석 100석인 회차에 1,000명이 동시에 선점 요청 시 정확히 100건만 성공하고 중복 선점·예약은 0건. 이 테스트는 대기열을 우회하는 `loadtest` 프로필에서 실행한다(10.2) 변경	동시성 통합 테스트, k6

NFR-02	정합성	1명이 같은 좌석에 여러 요청을 동시에 보내도 선점은 1건만 생성	동시성 통합 테스트
NFR-03	성능	좌석 선점 API p99 응답 시간 500ms 이하 (동시 사용자 1,000명)	k6
NFR-04	성능	좌석 조회 API p99 200ms 이하. 좌석·잔여석 조회는 API 서버별 로컬 캐시(Caffeine, TTL 1초)를 사용한다(FR-302·FR-303의 1초 지연 허용 범위)	k6
NFR-05	처리량	대기열 진입 초당 3,000건 처리 변경 MySQL 구현으로 달성한 처리량을 측정·기록하고, 목표에 못 미치면 Redis 도입 근거로 사용한다	k6
NFR-06	가용성	KOPIS 장애·지연 시에도 예매 기능은 정상 동작	외부 호출 차단 상태에서 예매 테스트
NFR-07	가용성	**보류** — Redis 도입 시 적용. 현재 구성에서는 5.1과 같이 Redis 의존 기능이 없다 변경	—
NFR-08	신뢰성	만료된 선점·주문은 만료 후 10초 이내에 좌석에 반영. 조회 시 지연 판정 + 5초 주기 스케줄러로 구현한다 변경	통합 테스트
NFR-09	보안	KOPIS 인증키, 토스 시크릿 키, JWT 서명 키, DB 비밀번호는 AWS SSM Parameter Store(SecureString)에 두고 환경변수로 주입하며 저장소에 커밋하지 않는다. 토스 클라이언트 키(`test_gck_`)만 Vue 빌드 환경변수로 노출한다 변경	코드 리뷰, secret scan
NFR-10	관측성	선점 성공·실패 건수, 락 대기 시간, API 응답 시간, 결제 승인 성공·실패·보정·보상 취소 건수를 메트릭으로 수집. Micrometer + Prometheus + Grafana 변경	대시보드 확인
NFR-11	정합성	모든 시점에서 토스 결제 상태 `DONE`인 주문 = 내부 `CONFIRMED` 주문이 되도록 보정·보상 취소로 수렴한다. 불일치 상태는 최대 2분 안에 해소된다 신규	Fake 게이트웨이 장애 주입 테스트, 대사 스크립트
NFR-12	가용성	토스페이먼츠 장애·지연 시 결제 외 기능(조회·대기열·선점)은 정상 동작하고, 결제는 FR-608·FR-609에 따라 실패 또는 보정 처리된다 신규	Fake 게이트웨이 지연·오류 주입
NFR-13	일관성	서버·DB 시간대는 Asia/Seoul로 통일하고, 만료 판정은 애플리케이션 서버 시각(NTP 동기화)을 기준으로 한다 신규	설정 리뷰

5.1 Redis 보류에 따른 구현 방식 변경

기능	현재 구현 (Redis 없음)	Redis 도입 시
대기열	MySQL `QueueEntry` 테이블 + ShedLock 스케줄러 (4.4)	Redis Sorted Set 구현으로 교체
입장 토큰	서명 JWT (저장소 불필요)	변경 없음
좌석 선점	전략 A·B·C (DB)	전략 D·E 추가 비교
좌석·잔여석 조회 캐시	API 서버별 로컬 캐시 (Caffeine, 1초)	필요하면 공유 캐시로 교체
멱등성·결제	DB 유니크 제약 + 토스 멱등키	변경 없음

---

6. 동시성·결제 정합성 시나리오 변경

각 시나리오는 테스트 케이스로 1:1 구현한다. CS-11 ~ CS-18은 Fake 결제 게이트웨이로 재현한다.

ID	상황	기대 결과
CS-01	사용자 A와 B가 같은 좌석을 동시에 선점 요청	한 명만 성공, 다른 한 명은 409
CS-02	1,000명이 100석에 동시에 선점 요청	정확히 100건 성공, 좌석당 선점 1건
CS-03	같은 사용자가 여러 탭에서 같은 좌석을 동시에 선점	1건만 생성, 선점 한도 계산이 중복되지 않음
CS-04	같은 사용자가 서로 다른 좌석 5개를 동시에 선점	최대 4건만 성공 (한도 검사의 경쟁 조건 확인)
CS-05	결제 기한 직전에 승인 요청, 동시에 만료 스케줄러 실행 변경	`PENDING`에서 `APPROVING`(승인)과 `EXPIRED`(만료) 중 조건부 전이에 먼저 성공한 쪽만 진행한다. 승인이 이기면 결제 성공 시 예약 확정, 만료가 이기면 토스를 호출하지 않고 410. 결제는 됐는데 좌석은 남이 가져가는 상황은 없다
CS-06	선점 만료와 동시에 다른 사용자가 같은 좌석 선점	만료 처리 후 새 선점 1건만 존재
CS-07	승인 성공 응답이 유실되어 클라이언트가 같은 `orderId`로 재요청 변경	토스 승인·예약은 1건만 생성, 재요청에 최초 결과를 돌려준다
CS-08	결제 실패(카드 거절 등) 확정	주문 `FAILED`. 원래 선점 만료 전이면 좌석은 `HELD`로 돌아가 같은 회원이 새 주문으로 재시도 가능, 만료 후면 `AVAILABLE`

CS-09	예약 취소와 동시에 다른 사용자가 해당 좌석 선점	토스 환불이 완료되어 좌석이 `AVAILABLE` 이 된 뒤에만 선점 가능, 상태 꼬임 없음
CS-10	대기열에서 같은 사용자가 동시에 여러 번 진입	순번 1개만 존재
CS-11	토스 승인은 성공했으나 예약 확정 트랜잭션 실패 신규	보상 취소가 실행되어 토스 결제는 취소, 주문 `FAILED` , 좌석은 CS-08 규칙대로 반환
CS-12	승인 API 타임아웃 (토스에서는 실제로 승인됨) 신규	주문은 `APPROVING` 으로 유지되고 만료 처리되지 않으며, 보정에서 `DONE` 을 확인해 예약 확정. 이중 결제 없음
CS-13	같은 주문에 두 탭에서 동시에 승인 요청 신규	토스 승인 호출은 1회, 한쪽은 확정 결과, 다른 쪽은 409 `PAYMENT_IN_PROGRESS` 또는 같은 확정 결과
CS-14	결제 인증 후 사용자가 이탈하여 승인 요청이 오지 않음 신규	결제 기한 후 주문 `EXPIRED` , 좌석 `AVAILABLE` . 승인되지 않았으므로 청구 없음
CS-15	클라이언트가 금액을 조작하여 승인 요청 신규	400 `AMOUNT_MISMATCH` , 토스 호출 없음, 주문은 `PENDING` 유지
CS-16	토스 상점관리자에서 결제를 취소하여 웹훅 수신 (중복 수신 포함) 신규	예약 `CANCELED` , 좌석 `AVAILABLE` 이 1번만 반영된다
CS-17	관리자 직권 취소와 회원 본인 취소가 동시에 요청됨 신규	`CONFIRMED → CANCELING` 조건부 전이에 성공한 한쪽만 진행한다. 토스 취소 호출 1회, 환불 1회, 다른 쪽은 409 `INVALID_STATE_TRANSITION`
CS-18	회차 일괄 취소 중 일부 건의 토스 취소가 실패하고, 실패 건을 재실행 신규	실패 건만 `CONFIRMED` 로 남고 재실행 시 그 건만 처리된다. 중복 환불 0건, 작업 종료 후 해당 회차의 `CONFIRMED` 예약 0건

---

7. 기술적 의사결정

7.1 좌석 선점 동시성 제어 후보

전략 A·B·C를 구현하고, 같은 부하 조건에서 비교한 뒤 최종 전략을 결정한다. 전략 D·E는 Redis 도입이 결정되면 같은 조건으로 추가 비교한다. 선점 API는 전략을 설정으로 바꿔 실행할 수 있도록 설계한다. 주문 생성 이후(`PAYING` 부터)의 상태 전이는 전략과 관계없이 DB 조건부 UPDATE로 처리한다.

전략	방식	예상 장점	예상 단점
A. 비관적 락	`SELECT ... FOR UPDATE`	구현 단순, 정합성 확실	경합 시 DB 커넥션 대기, 처리량 저하
B. 낙관적 락	`@Version` 버전 비교	경합 적을 때 빠름	경합 심하면 실패·재시도 폭증
C. 조건부 UPDATE	`UPDATE seat SET status='HELD' WHERE id=? AND status='AVAILABLE'` 후 영향 행 수 확인	락 없이 원자적, 단순	다중 좌석 원자성 처리 필요
D. Redis 분산 락 **(보류)**	Redisson 락 후 DB 반영	DB 부하 분산	락 획득·해제 비용, Redis 의존. 서버가 1대인 현재 구성에서는 비교 의미가 작음
E. Redis 원자 연산 **(보류)**	`SET key value NX EX 300` 으로 선점 (다중 좌석은 Lua 스크립트로 원자 처리)	매우 빠름, TTL로 만료 자동 처리	Redis와 DB 간 정합성 관리 필요. 선점 단계의 기준 저장소가 Redis이므로 좌석 조회 시 Redis 상태를 합쳐야 함

비교 지표: 성공 건수 정확성, 처리량(TPS), p50/p95/p99 응답 시간, DB CPU·커넥션 사용량, 실패 응답 비율

7.2 기타 결정 사항 변경

주제	후보	방향
선점 만료 처리	Redis TTL / DB 만료 시각 + 스케줄러	DB `expiresAt` + 조회 시 지연 판정 + 5초 주기 스케줄러 (ShedLock으로 한 서버에서만 실행)
대기열	MySQL 테이블 / Redis Sorted Set / Kafka	MySQL 테이블 (Redis 보류). 인터페이스로 분리해 Redis 도입 시 교체
입장 토큰	Redis 저장 토큰 / 서명 JWT	서명 JWT (별도 저장소 없이 검증 가능)
멱등성	Idempotency-Key 헤더 + Redis 결과 저장 / `orderId` 기반 DB 상태	`orderId` 를 멱등 기준으로 하고 주문 상태와 DB 유니크 제약(`orderId` , `paymentKey`)으로 보장. 토스 호출에는 `Idempotency-Key` 헤더 사용
결제 연동	Mock / 토스페이먼츠	토스페이먼츠 결제위젯 v2 + 승인 API. 게이트웨이 인터페이스로 분리하고 테스트·부하 테스트는 Fake 구현
외부 데이터	실시간 호출 / 배치 적재	배치 적재 (NFR-06)

7.3 결정 기록 (v0.3 이후 확정) 신규

결정	내용과 근거
다중 좌석 선점은 전부 성공 또는 전부 실패 (FR-504)	일부만 선점되면 사용자가 원치 않는 좌석 조합이 남고 한도 계산이 복잡해진다. v0.2 본문과 미결 사항의 충돌을 본문 기준으로 정리
결제 실패 시 선점 유지 후 재시도 허용 (CS-08)	카드 거절은 다른 카드로 즉시 재시도하는 경우가 많다. 단, 원래 선점 만료 시각을 넘겨 연장하지는 않아 좌석이 무한정 묶이지 않는다. v0.2 상태 전이도(즉시 해제)를 이 결정에 맞게 수정
좌석 상태 `PAYING` 도입 (CS-05)	결제 인증 화면에서 시간이 걸려도 선점 만료로 좌석이 다른 사람에게 넘어가지 않게 한다. 결제 기한(10분)은 토스 결제 인증 유효 시간과 같다
대기열 우회 프로필 (NFR-01)	활성 사용자 N=100이면 실제 흐름에서 1,000명 동시 선점이 일어나지 않는다. 선점 로직의 한계를 측정하기 위해 `loadtest` 프로필에서만 테스트용 입장 토큰을 일괄 발급한다. 운영 프로필에서는 비활성
선점 한도에 예약 좌석 포함 (FR-503)	포함하지 않으면 4석 예약 후 다시 4석을 선점하는 방식으로 한도가 무력화된다
멱등 저장소를 Redis에서 DB로 이동	결제는 캐시성 저장소에 의존하지 않아야 하고, 토스 멱등키 보관 기간(15일)이 v0.2의 Redis 24시간보다 길다
전체 취소만 지원, 가상계좌 제외	부분 취소와 입금 대기 상태는 좌석 상태 전이를 크게 늘린다. 동시성 검증이라는 목적에 비해 비용이 크다
선점·해제 API를 좌석 ID 기준으로 변경	전략 E는 DB에 Hold 행을 만들지 않으므로 hold ID 대신 좌석 ID로 다뤄야 전략과 관계없이 같은 API를 쓸 수 있다
대기열 활성 사용자 수 N (FR-403)	$N = \max(10, \min(N_{성능}, N_{좌석}))$ · $N_{성능}$ = 최종 전략에서 선점 API p99 500ms(NFR-04)를 만족한 최대 동시 사용자 수 × 0.7 (30% 여유). 측정 전에는 100 · $N_{좌석}$ = ⌈잔여 좌석 수 ÷ 1인 평균 구매 좌석 수(초기 2) × 1.5⌉. 살 수 있는 좌석보다 훨씬 많은 인원을 들여보내면 대부분 실패만 겪으므로 상한을 둔다 근거: 활성 사용자 수는 서버가 감당할 수 있는 동시 처리량과 실제 남은 좌석 중 작은 쪽에 맞춰야 한다. 1인 평균 구매 좌석 수는 운영 데이터가 쌓이면 갱신한다. 관리자가 회차별 상한을 지정하면(FR-910) 산정 결과와 지정값 중 작은 값을 쓴다
예약 취소 기한 (FR-606)	회차 시작 24시간 전까지, 설정값으로 조정 가능. 직전 취소를 허용하면 되팔 시간이 없는 좌석이 생기고, 국내 예매처도 관람 전일까지만 취소를 허용하는 경우가 일반적이다. 수수료 정책은 범위 밖이므로 기한 안에서는 전액 환불
KOPIS 출처 표기 (FR-109)	공공데이터포털에 등록된 KOPIS 공연목록 API의 이용허락범위는 "제한 없음"이지만, 데이터 출처를 밝히는 것이 공공데이터 이용의 일반 관행이므로 응답·화면·README에 모두 표기한다
KOPIS 호출량 (FR-110)	공식 일일 한도가 공개되어 있지 않아(기관 정책에 따라 다름) 보수적인 예산(1,000회)과 속도 제한을 설정값으로 둔다. 일일 수집 호출 수 추정: 기간 210일 ÷ 31일 = 7구간 × 구간당 수 페이지 + 신규 공연 상세·시설 상세. 초기 적재일을 빼면 예산 안에 들어간다. 첫 전체 적재는 여러 날에 나눠 진행될 수 있다
부하 테스트 환경 (10.2) 변경	AWS에서 실행한다. 실제 배포 구성(EC2 1대 + Caddy + RDS)과 같은 조건에서 측정해야 결과를 그대로 쓸 수 있다. 다만 dev의 t3 인스턴스는 CPU 크레딧 소진 시 성능이 떨어져 결과가 흔들리므로, 부하 테스트에서는 같은 구성을 성능 고정형(c7i) 인스턴스로 만든다. 테스트가 끝나면 Terraform으로 삭제하여 비용을 관리한다. v0.4의 로컬 Docker Compose 결정을 대체
Redis 보류 처리 신규	Redis 없이 대기열은 MySQL, 캐시는 로컬 캐시로 구현하고, Redis에만 의존하는 전략 D·E와 Redis 장애 요구사항(NFR-07)은 보류한다. 대기열 처리량(NFR-05)과 선점 성능을 측정하여 목표 미달 시 Redis 도입을 결정한다. 인프라 구성 요소를 늘리지 않고 시작하되, 도입 판단을 수치로 하기 위함
EC2 1대 + Caddy 배포 신규	3인 팀의 학습·운영 부담과 비용을 줄이기 위해 컨테이너 오케스트레이션(ECS)과 로드 밸런서 없이 EC2 1대로 배포한다. Caddy는 설정이 짧고 HTTPS를 자동으로 처리하며, 프론트와 API를 같은 도메인으로 묶어 CORS와 쿠키·리다이렉트 문제를 없앤다. 서버가 1대이므로 무중단 배포와 수평 확장은 범위 밖이며, ShedLock·로컬 캐시 설계는 나중에 서버를 늘려도 그대로 동작한다
관리자 화면 포함 (v0.9) 변경	v0.5에서는 관리자 화면을 빼고 Swagger로 대체했으나, 공연 취소 시 예약 처리(FR-108), 결제 이상 건 처리처럼 운영자가 실제로 손써야 하는 일이 있어 화면까지 포함한다. 프론트 작업량을 줄이기 위해 관리자 화면에만 Element Plus를 쓰고, P2 기능은 일정에 따라 뒤로 미룬다. 관리자가 주문·결제 상태를 직접 덮어쓰는 기능은 정합성을 깨뜨릴 수 있어 두지 않고, 보정·재시도만 실행하게 한다

8. 데이터 모델 변경

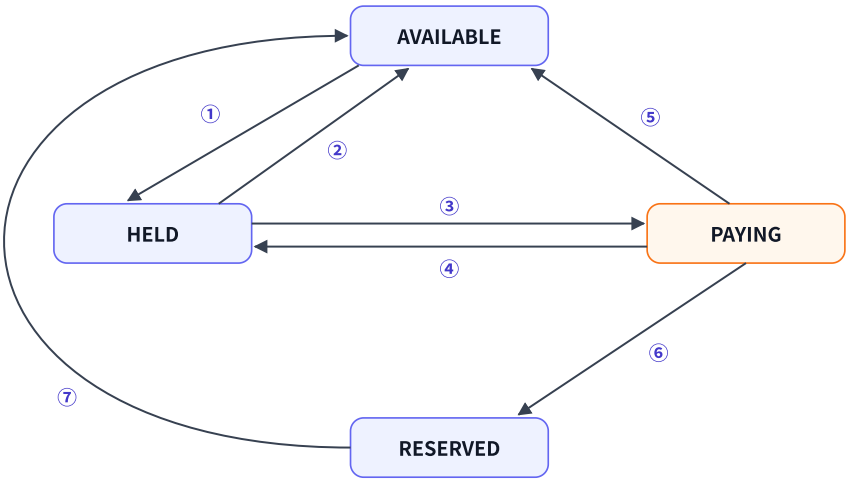
8.1 주요 엔티티

엔티티	주요 필드
Performance	id, kopisId(mt20id), name, genre, area, startDate, endDate, posterUrl, runtime, facilityId, status(공연예정/공연중/공연완료), **saleStatus**(ON_SALE / NO_SCHEDULE / NOT_FOR_SALE / SALE_STOPPED), **detailSyncedAt**, syncedAt
Facility	id, kopisId(mt10id), name, address, seatCount, latitude, longitude

PriceGrade	id, performanceId, gradeName, price, **sortOrder**
Schedule	id, performanceId, startAt, openAt, **autoGenerated**, **queueLimitOverride**(관리자 지정 N 상한, null 허용)
Seat	id, scheduleId, rowName, number, gradeId, status(AVAILABLE / HELD / **PAYING** / RESERVED), version
Hold	id, seatId, memberId, scheduleId, expiresAt, **status**(ACTIVE / RELEASED / EXPIRED / CONVERTED). 전략 E에서는 생성하지 않는다
Reservation (주문)	id, **orderId**, memberId, scheduleId, status(PENDING / APPROVING / CONFIRMED / CANCELING / CANCELED / FAILED / EXPIRED), totalPrice, **paymentDeadline**, **failureCode**, **canceledBy**(MEMBER / ADMIN / WEBHOOK), **cancelReason**, createdAt, confirmedAt, canceledAt
ReservationSeat	reservationId, seatId, **gradeName**, **price** (주문 시점 가격 스냅샷)
Payment	id, reservationId, **paymentKey**, **orderId**, **method**, status(토스 상태: DONE / CANCELED / ABORTED / EXPIRED 등), amount, approvedAt, canceledAt, **rawResponse**(JSON)
PaymentEvent	id, orderId, type(CONFIRM_REQUEST / CONFIRM_RESULT / CANCEL / WEBHOOK / RECONCILE), payload, createdAt — 결제 관련 외부 통신 이력(감사·디버깅용)
QueueEntry	id(자동 증가 = 순번), scheduleId, memberId, status(WAITING / ACTIVE), enteredAt, activatedAt, tokenExpiresAt. `(scheduleId, memberId)` 유니크. 이탈·만료된 행은 삭제하여 재진입 시 맨 뒤 순번을 받는다 신규
Member	id, email, passwordHash, **role**(USER / ADMIN), **status**(ACTIVE / SUSPENDED), **suspendReason**, **customerKey**, createdAt
AdminAuditLog	id, adminId, action, targetType, targetId, reason, beforeValue(JSON), afterValue(JSON), createdAt — 수정·삭제 불가(애플리케이션 계정에 UPDATE·DELETE 권한 미부여) 신규
BulkCancelJob	id, scheduleId, reason, status(RUNNING / COMPLETED / COMPLETED_WITH_FAILURES), totalCount, succeededCount, failedCount, createdBy, createdAt, finishedAt 신규
SyncHistory	id, startedAt, finishedAt, fetchedCount, newCount, **updatedCount**, failedCount, **apiCallCount**
SyncFailure	id, syncHistoryId, requestType, requestParams, errorMessage, createdAt

유니크 제약: `Seat(scheduleId, rowName, number)`, `Schedule(performanceId, startAt)`, `Reservation.orderId`, `Payment.paymentKey`, `Member.email`, `Member.customerKey`. Hold는 좌석당 `ACTIVE` 1건만 존재해야 한다(활성 여부를 나타내는 생성 컬럼 + 유니크 인덱스 등으로 보장).

8.2 좌석 상태 전이



#	전이	조건
①	AVAILABLE → HELD	선점 (FR-501)
②	HELD → AVAILABLE	직접 해제 (FR-505), 선점 만료 (FR-502)
③	HELD → PAYING	주문 생성 = 결제 시작 (FR-607)
④	PAYING → HELD	결제 실패, 원래 선점 만료 시각 전 (FR-608)
⑤	PAYING → AVAILABLE	결제 실패(선점 만료 후), 결제 기한 만료 (FR-602), 보상 취소 (FR-610)
⑥	PAYING → RESERVED	결제 승인 및 예약 확정 (FR-601)
⑦	RESERVED → AVAILABLE	예약 취소 후 토스 환불 완료 (FR-606, FR-907), 외부 취소 웹훅 반영 (FR-611)

허용되지 않은 전이(예: AVAILABLE → RESERVED, HELD → RESERVED)는 거절한다(409 `INVALID_STATE_TRANSITION`). 모든 전이는 현재 상태를 조건으로 하는 UPDATE(또는 선택한 동시성 전략)로 수행한다.

8.3 주문(Reservation) 상태 전이 신규

전이	조건
(생성) → PENDING	주문 생성 (FR-607)
PENDING → APPROVING	금액·기한 검증 통과 후 승인 시작 (FR-601)
PENDING → FAILED	failUrl 리다이렉트 등 승인 전 실패 (FR-608)
PENDING → EXPIRED	결제 기한 경과 (FR-602)
APPROVING → CONFIRMED	승인 성공 및 내부 반영 성공, 또는 보정에서 DONE 확인 (FR-601, FR-609)
APPROVING → FAILED	승인 거절, 보정에서 미승인 확인, 보상 취소 (FR-608 ~ FR-610)
CONFIRMED → CANCELING → CANCELED	회원 취소(FR-606) 또는 관리자 직권·일괄 취소(FR-907) 후 토스 취소 성공. 토스 취소 실패 시 CANCELING → CONFIRMED
CONFIRMED → CANCELED	외부 취소 웹훅 반영 (FR-611)

---

9. API 명세 개요 변경

Method	URL	설명	관련 FR
GET	/api/performances	공연 목록 (출처 표기 포함)	FR-301, FR-109
GET	/api/performances/{id}	공연 상세 + 회차	FR-302
GET	/api/schedules/{id}/seats	좌석 배치·상태	FR-303
POST	/api/schedules/{id}/queue	대기열 진입	FR-401, FR-405
GET	/api/schedules/{id}/queue/me	내 순번·예상 대기 시간 조회 (입장 시 토큰, 매진 시 soldOut 포함)	FR-402, FR-403, FR-407
DELETE	/api/schedules/{id}/queue/me	대기열 이탈	FR-406
POST	/api/schedules/{id}/holds	좌석 선점 (헤더: 입장 토큰, body: seatIds)	FR-501 ~ FR-504
DELETE	/api/schedules/{id}/holds	선점 해제 (body: seatIds)	FR-505
POST	/api/orders	주문 생성 (헤더: 입장 토큰, body: scheduleId, seatIds) → orderId, amount, orderName, customerKey	FR-607
POST	/api/payments/confirm	결제 승인 (body: paymentKey, orderId, amount)	FR-601 ~ FR-603, FR-609, FR-610
POST	/api/payments/fail	결제 실패 통지 (body: orderId, code, message)	FR-608
POST	/api/webhooks/toss	토스 웹훅 수신 (인증 없음, 조회 API로 재검증)	FR-611
GET	/api/orders/{orderId}	주문 상태 조회 (결제 결과 대기 시 폴링)	FR-609, FR-805
GET	/api/reservations/me	내 예약 목록	FR-605
DELETE	/api/reservations/{id}	예약 취소 (토스 환불)	FR-606
POST	/api/auth/signup, /api/auth/login	회원가입, 로그인	FR-701
관리자 API (모두 `ADMIN` 역할 필요, 변경 작업은 body에 `reason` 포함 및 감사 로그 기록 FR-912)			
GET	/api/admin/dashboard	대시보드 집계	FR-902
GET	/api/admin/performances	공연 검색	FR-903
PATCH	/api/admin/performances/{id}	판매 상태 변경, 가격 등급 수정	FR-903
POST	/api/admin/performances/{id}/schedules	회차 등록	FR-201, FR-904
PATCH	/api/admin/schedules/{id}	오픈 시각·일시 수정, N 상한 지정	FR-204, FR-904, FR-910

DELETE	/api/admin/schedules/{id}	회차 삭제 (선점·예약 없을 때)	FR-904
GET	/api/admin/schedules/{id}/seats	좌석 현황 (선점자·주문 포함)	FR-905
GET	/api/admin/reservations	예약·결제 검색	FR-906
GET	/api/admin/reservations/{id}	예약 상세 + 결제 통신 이력	FR-906
POST	/api/admin/reservations/{id}/cancel	직권 취소	FR-907
POST	/api/admin/schedules/{id}/bulk-cancel	회차 일괄 취소 시작	FR-907
GET / POST	/api/admin/bulk-cancel-jobs/{id}, …/{id}/retry	일괄 취소 진행률 조회, 실패 건 재실행	FR-907
GET	/api/admin/payment-issues	결제 이상 건 목록	FR-908
POST	/api/admin/orders/{orderId}/reconcile, …/compensate	토스 상태 재조회, 보상 취소 재시도	FR-908
POST	/api/admin/sync	KOPIS 재동기화	FR-107, FR-909
GET	/api/admin/sync-histories, …/{id}/failures	동기화 이력, 실패 건	FR-909
POST	/api/admin/sync-failures/{id}/retry	실패 건 재시도	FR-909
GET	/api/admin/queues	회차별 대기열 현황	FR-910
GET / PATCH	/api/admin/members, /api/admin/members/{id}	회원 검색·상세, 정지·해제	FR-911
GET	/api/admin/audit-logs	감사 로그 조회	FR-912

9.1 주요 오류 코드 변경

HTTP	코드	상황
400	INVALID_REQUEST	잘못된 파라미터
400	AMOUNT_MISMATCH	승인 요청 금액이 주문 금액과 다름
400	PAYMENT_FAILED	토스 승인 거절 (응답에 토스 오류 코드·메시지 포함)
401	UNAUTHORIZED	미로그인
403	FORBIDDEN_RESOURCE	본인 소유가 아닌 선점·주문·예약 접근, 관리자 API 권한 없음
403	QUEUE_TOKEN_REQUIRED	입장 토큰 없음·만료
403	SALE_NOT_OPEN	예매 오픈 전, 판매 중지·판매 불가 공연
403	MEMBER_SUSPENDED	정지된 회원의 로그인·대기열·선점 요청
403	CANCEL_PERIOD_EXPIRED	취소 기한(회차 시작 24시간 전) 경과
404	NOT_FOUND	공연·회차·좌석·주문 없음
409	SEAT_ALREADY_TAKEN	이미 선점·결제 중·예약된 좌석
409	HOLD_LIMIT_EXCEEDED	회차당 보유 한도(4석) 초과
409	INVALID_STATE_TRANSITION	허용되지 않은 상태 전이 (예: 결제 중 좌석 해제, 이미 취소된 예약 취소)
409	PAYMENT_IN_PROGRESS	같은 주문의 승인이 처리 중
409	BULK_CANCEL_IN_PROGRESS	같은 회차의 일괄 취소 작업이 이미 실행 중
409	RESOURCE_IN_USE	선점·예약이 있는 회차의 일시 수정·삭제, 가격 등급 수정
409	SYNC_IN_PROGRESS	동기화가 이미 실행 중
410	HOLD_EXPIRED	선점 만료 후 주문 생성 시도
410	ORDER_EXPIRED	결제 기한 경과 후 승인 시도
502	PAYMENT_GATEWAY_ERROR	토스 취소 실패 등 PG 오류 (승인 결과 불명은 202로 응답하고 보정 진행)
503	TEMPORARILY_UNAVAILABLE	DB 등 의존 시스템 장애

승인 결과가 불명확할 때(FR-609) `/api/payments/confirm` 은 202와 주문 상태 `APPROVING` 을 응답하고, 클라이언트는 주문 상태를 폴링한다.

---

10. 테스트 및 검증 계획

10.1 동시성 통합 테스트

- `ExecutorService` 와 `CountDownLatch` 로 N개 스레드가 동시에 요청을 시작하도록 구성한다.
- CS-01 ~ CS-18을 각각 테스트 케이스로 작성한다. CS-11 ~ CS-18은 Fake 결제 게이트웨이의 장애 주입 설정으로 재현한다.
- 검증은 응답 코드뿐 아니라 최종 DB 상태(좌석별 선점·예약 건수, 주문 상태, Fake 게이트웨이의 승인·취소 호출 횟수)로 한다.
- 테스트 DB(MySQL 8)는 Testcontainers로 실제와 같은 환경에서 실행한다.

10.2 부하 테스트 (k6) 변경

부하 테스트는 `loadtest` 프로필에서 실행한다. 이 프로필은 Fake 결제 게이트웨이를 쓰고, 테스트용 입장 토큰 일괄 발급 API를 활성화한다(운영 프로필에서는 비활성). 토스 샌드박스는 부하 테스트에 사용하지 않는다.

테스트	조건	측정
오픈 스파이크	100석, 가상 사용자 1,000명 동시 선점	성공 건수, 중복 건수, p99
전략 비교	7.1의 A·B·C 전략을 같은 조건으로 각각 실행 (D·E는 Redis 도입 시)	TPS, p50/p95/p99, DB 부하
대기열	초당 3,000건 진입 (MySQL 구현)	처리량, 오류율, DB CPU
예매 전 구간	선점 → 주문 → 승인(Fake, 지연 200ms) 흐름을 500명이 동시에 수행	확정 건수 = 승인 건수, p99
장애 주입	Fake 게이트웨이 타임아웃 비율 10%, 테스트 도중 애플리케이션 컨테이너 재시작 (진행 중 주문의 보정 확인)	오류 응답 형태, 데이터 정합성(NFR-11)

테스트 환경 (AWS, 서울 리전, 같은 AZ) 변경

배포 구성과 같은 형태를 Terraform으로 테스트할 때만 생성하고 끝나면 삭제한다. 요청은 k6 → Caddy(HTTPS) → Spring Boot 경로로 보내 실제 경로와 같게 측정한다. 각 테스트는 3회 실행하여 중앙값을 기록하고, 실제 사용한 사양을 README에 기록한다.

구성 요소	사양 (초기값)
애플리케이션 서버	EC2 c7i.xlarge 1대 (4 vCPU / 8 GB): Caddy + Spring Boot 컨테이너 (JVM 힙 4 GB), HikariCP 최대 풀 20
MySQL 8	RDS db.m7g.large (2 vCPU / 8 GB), gp3 스토리지, 단일 AZ, `max_connections` 200
k6	EC2 c7i.xlarge 1대 (4 vCPU), 애플리케이션 서버와 같은 VPC·AZ
모니터링	Prometheus + Grafana EC2 t3.medium 1대, RDS는 Performance Insights 병행
데이터	공연 1,000건, 회차 10,000건, 대상 회차 좌석 100석 / 1,000석 두 가지

10.3 토스페이먼츠 연동 테스트 신규

- 토스 테스트 키로 결제위젯 → 승인 → 예약 확정 → 취소(환불) 정상 흐름을 E2E로 1회 이상 검증한다.
- 테스트 환경의 카드 거절·인증 실패 시나리오로 failUrl 처리(FR-608)를 검증한다.
- 웹훅은 dev 환경 도메인(`https://{도메인}/api/webhooks/toss`)을 토스 개발자센터에 등록하여 수신을 확인한다. 로컬에서 확인할 때만 터널링 도구를 쓴다.
- Vue 결제 화면의 성공·실패 리다이렉트(FR-804 ~ FR-806)를 dev 환경에서 브라우저로 확인한다.

10.4 결과 기록

전략별 결과를 표와 그래프로 정리하여 README에 게시하고, 최종 전략 선택 근거를 작성한다. 목표 미달 시 원인 분석과 개선 과정을 함께 기록한다.

---

11. 프로젝트 완료 기준 변경

단계를 나누지 않고 4장의 모든 기능 요구사항을 한 번에 구현한다. 아래 조건을 모두 만족하면 프로젝트를 완료한 것으로 본다.

영역	완료 기준
기능	4장의 기능 요구사항(FR-101 ~ FR-912)이 모두 구현되고 인수 기준을 통과한다
정합성	6장의 시나리오(CS-01 ~ CS-18)가 모두 테스트로 작성되어 통과한다
성능·가용성	5장의 비기능 요구사항(NFR-01 ~ NFR-13, 보류된 NFR-07 제외)을 검증하고 결과를 기록한다
결제 연동	10.3의 토스페이먼츠 E2E 테스트(승인·실패·취소·웹훅)가 통과한다
전략 비교	7.1의 전략 A·B·C를 같은 조건에서 부하 테스트하고 최종 전략과 선택 근거, Redis 도입 여부 판단을 문서화한다
배포	dev 환경(EC2 + Caddy, HTTPS)이 `main` 브랜치에서 자동 배포되고, loadtest 환경을 Terraform으로 생성·삭제할 수 있다
문서	README에 아키텍처, 실행 방법(환경변수 목록 포함), 테스트 결과, 의사결정 기록을 정리한다

---

12. 팀 구성과 협업 신규

12.1 역할 분담 (확정)

담당	책임 범위	주요 ID
서창훈 PM · 인프라	일정·이슈 관리와 명세서 관리(12.4). AWS(Terraform), GitHub Actions CI/CD, 모니터링. 백엔드 중 독립적인 영역인 KOPIS 동기화 배치와 좌석·회차 데이터 생성, 그리고 이와 연결된 관리자 API(대시보드 집계, 공연·회차 관리, 동기화 관리, 회원 관리, 감사 로그). k6 부하 테스트 시나리오 작성·실행과 결과 정리	3.1, FR-1xx, FR-2xx, 관리자 API 중 FR-902 ~ FR-904, FR-909, FR-911, FR-912, NFR-09, NFR-10, 10.2
조은혜 백엔드	조회 API, 대기열·입장 토큰, 선점과 동시성 전략 A·B·C, 주문·결제(토스 승인·취소·웹훅·보정·보상 취소, Fake 게이트웨이), 회원·JWT. 결제·예약과 연결된 관리자 API(좌석 현황, 예약·결제 조회, 직권·일괄 취소, 결제 이상 건, 대기열 모니터링). 동시성·결제 정합성 테스트	FR-3xx ~ FR-7xx, FR-905 ~ FR-908, FR-910, CS-01 ~ CS-18, NFR-01 ~ NFR-04, NFR-08, NFR-11 ~ NFR-13
김민수 프론트엔드	Vue 사용자 앱과 토스 결제위젯 연동, 관리자 화면 전체(Element Plus), API 연동 목 데이터, dev 환경 브라우저 E2E 확인	FR-801 ~ FR-807, FR-901 ~ FR-912 화면, 10.3 화면 확인

> **작업량 주의**: 핵심 백엔드(동시성·결제)가 한 사람에게 몰리고, 관리자 화면 추가로 프론트 화면 수가 약 2배가 된다. 그래서 백엔드 중 다른 코드와 분리된 영역과 그에 딸린 관리자 API는 서창훈이 맡는다. 일정이 밀리면 아래 순서로 조정한다.
>
> - 관리자 P2 기능(FR-902 대시보드, FR-910 대기열 모니터링, FR-911 회원 관리, FR-912 조회 화면)을 뒤로 미룬다. 감사 로그 기록 자체는 유지한다.
> - 전략 A·B 구현을 뒤로 미룬다. 전략 C만으로도 전체 흐름과 정합성 검증은 완료할 수 있다.
> - FR-205(회차 자동 생성)를 관리자 수동 등록(FR-201)으로 대체한다.
> - 김민수가 프론트 작업 후 남는 시간에 백엔드 API 통합 테스트 작성을 돕는다.

12.2 공통 규칙

항목	규칙
저장소	모노레포 하나: `backend/`, `frontend/`, `infra/` (Terraform), `loadtest/` (k6), `docs/` (명세서, ADR)
브랜치	GitHub Flow. `main` 보호. 기능 브랜치 이름에 Jira 이슈 키를 넣는다: `feat/{Jira 키}-{요약}` (예: `feat/KAN-41-hold`). 커밋 메시지와 PR 제목에도 Jira 키를 넣어 Jira 이슈에 브랜치·커밋·PR이 자동으로 연결되게 한다
리뷰	모든 PR은 1명 이상 승인. 백엔드 PR은 서창훈이 리뷰하고, 결제·상태 전이·동시성 PR은 CS 테스트 통과를 병합 조건으로 한다. 프론트 PR 중 API 연동 부분은 조은혜가 함께 본다
API 계약	구현 전에 springdoc-openapi로 요청·응답 스키마를 먼저 PR로 합의한다. 프론트는 합의된 스키마로 목 데이터를 두고 병행 개발한다
응답 형식	오류: `{ "code", "message", "details" }` (9.1 코드 사용). 시각: ISO-8601 `+09:00`. 금액: 정수(원). ID: 숫자, 토스 `orderId` 만 문자열
CI	GitHub Actions: 백엔드 빌드·테스트(Testcontainers MySQL), 프론트 lint·빌드, gitleaks 비밀값 검사. 통과해야 병합 가능
DB 스키마	Flyway 마이그레이션으로 관리. 마이그레이션 파일은 번호 충돌을 막기 위해 PR 병합 순서대로 번호를 붙인다
문서	요구사항 변경은 이 명세서의 버전과 변경 이력을 올려 PR로 반영한다. 설계 결정은 `docs/adr/` 에 ADR로 남긴다

12.3 작업 순서

순서	서창훈 (PM·인프라)	조은혜 (백엔드)	김민수 (프론트)

1. 공통	저장소·CI 구성, 엔티티와 Flyway 초기 스키마, 오류 응답 형식, OpenAPI 초안을 셋이 함께 확정. 서창훈이 Jira에 요구사항 ID별 이슈를 만들고 Slack 채널을 구성한다		
2	도메인 준비, dev 환경 Terraform(EC2, RDS, 보안 그룹, IAM), Docker Compose·Caddyfile, CI/CD 파이프라인	회원·JWT, 공연·좌석 조회 API, 주문 엔티티·상태 전이	Vue 프로젝트 구성, 로그인·공연 목록·상세 화면(목 데이터)
3	회차 등록·좌석 생성(FR-201 ~ 204), dev 자동 배포 완성	선점(전략 C), 주문 생성, Fake 게이트웨이로 승인·실패·취소 전 구간 연결	좌석 선택·결제 화면, 실제 API 연결
4	KOPIS 동기화(FR-101 ~ 110), FR-205	CS-01 ~ CS-09 테스트, 토스 실제 연동·웹훅·보정·보상 취소, CS-11 ~ 16 테스트	토스 결제위젯 연결, 성공·실패 화면, dev에서 E2E 확인
5	관리자 API: 공연·회차 관리, 동기화 관리, 감사 로그 (P1)	대기열·입장 토큰, CS-10, 관리자 API: 좌석 현황, 예약·결제 조회, 직권·일괄 취소, 결제 이상 건, CS-17·18	대기열·내 예약 화면, 관리자 레이아웃과 P1 관리자 화면
6	loadtest 환경·모니터링, k6 시나리오 작성, 관리자 P2 API(대시보드, 회원 관리)	전략 A·B 추가, 관리자 P2 API(대기열 모니터링)	관리자 P2 화면, 화면 다듬기
7. 공통	부하 테스트 실행(서창훈 주도, 조은혜 원인 분석·개선), Redis 도입 여부 판단, README 정리		

12.4 PM 업무 (서창훈)

항목	내용
이슈 관리	Jira 스크럼 보드(1주 스프린트)에서 관리한다. 구조는 12.5를 따른다. 이슈 제목에 FR·CS·NFR ID를 붙이고, 스프린트 시작 전에 다음 스프린트 이슈와 담당자를 확정한다
회의	주 1회 스프린트 점검(완료·진행·막힌 것, 다음 스프린트 계획), Slack 허들로 진행. 평일에는 `#daily` 에 비동기 데일리(어제 한 일·오늘 할 일·막힌 것 3줄)
명세서 관리	요구사항 변경 요청을 받아 명세서 버전을 올리고 변경 이력을 남긴다. 변경 사항은 팀에 공지한다
결정 기록	회의에서 정한 설계 결정을 `docs/adr/` 에 남기고 명세서 7.3과 맞춘다
위험 관리	백엔드 일정 지연을 가장 큰 위험으로 보고, 매주 진행률을 확인해 12.1의 조정 순서를 적용할지 판단한다
외부 준비	토스 테스트 키, AWS 계정·IAM·비용 알림, dev 도메인 준비 (13장). KOPIS 인증키는 조은혜가 준비하고 PM은 진행만 확인

12.5 협업 도구 (Jira, Slack) 신규

Jira 구성

항목	규칙
프로젝트	프로젝트 "예매한 사이"(키 `KAN`, https://oso7865.atlassian.net). 1주 단위로 운영한다. 작업 순서(12.3)의 단계를 스프린트에 나눠 배치한다
에픽	명세서 4장의 기능 영역 단위: KOPIS 연동(FR-1xx), 데이터 생성(FR-2xx), 조회(FR-3xx), 대기열(FR-4xx), 선점(FR-5xx), 주문·결제(FR-6xx), 회원(FR-7xx), 사용자 화면(FR-8xx), 관리자(FR-9xx), 인프라, 부하 테스트
이슈	요구사항 ID 1개 = 스토리 1개가 기본. 백엔드·프론트 작업이 모두 필요한 기능은 하위 작업으로 나눈다. CS 시나리오는 해당 기능 스토리의 하위 작업 또는 별도 테스트 작업으로 등록
필드	컴포넌트: `BE` / `FE` / `INFRA`. 라벨: `P1` / `P2`. 우선순위 P2는 일정 조정 시 다음 스프린트로 넘길 대상
상태	할 일 → 진행 중 → 리뷰 중(PR 생성) → 완료. "완료"는 인수 기준 충족, PR 병합, CI 통과, 관련 CS 테스트 통과를 모두 만족할 때만
GitHub 연동	GitHub for Jira 앱을 설치해 브랜치·커밋·PR이 Jira 키로 이슈에 자동 연결되게 한다

Slack 채널

채널	용도
`#notice`	공지, 명세서 버전 변경, 회의 결과 (PM만 작성)
`#dev`	개발 논의 전반. 백엔드·프론트 API 협의도 여기서 하고, 결정된 내용은 스레드에 정리해 Jira 이슈에 링크
`#daily`	비동기 데일리 스크럼
`#github`	GitHub 앱 알림: PR 생성·리뷰 요청·병합, Actions 빌드·배포 결과
`#jira`	Jira Cloud 앱 알림: 이슈 생성·담당자 지정·상태 변경

`#alerts`	AWS 알람(CloudWatch → SNS → Slack), 보상 취소 실패·보정 지연 알림. 이 채널의 알림은 담당자가 스레드로 처리 결과를 남긴다

---

13. 미결 사항 변경

v0.6 기준 미결 사항은 없다. v0.2 ~ v0.3의 미결 사항은 모두 7.3에서 확정했다.

구현 착수 시 확인할 외부 사항 (확인 결과에 따라 설정값만 바꾸면 되도록 설계에 반영해 둠)

- (담당: 조은혜) KOPIS 인증키 발급 후 공식 가이드·공통코드 문서로 엔드포인트, 파라미터, 최대 조회 기간, 일일 호출 한도를 확인하고 FR-101·FR-110의 설정값을 맞춘다. KOPIS 호출부는 어댑터로 분리하고, 실제 응답 샘플로 계약 테스트를 작성한다.
- 토스페이먼츠 개발자센터에서 테스트 키를 발급하고 결제위젯의 결제수단 노출 설정(카드·간편결제만, FR-612)을 적용한다.
- AWS 계정과 IAM 권한을 팀원별로 준비하고, 비용 알림(AWS Budgets)을 설정한다.
- dev 환경용 도메인을 준비한다(Caddy HTTPS 자동 발급과 토스 웹훅 수신에 필요). 준비 전까지는 3.1의 임시 운영 방식을 따른다. 늦어도 토스 실제 연동(12.3의 4단계) 전에 준비한다.

---

14. 참고

- KOPIS 공연예술통합전산망 오픈API 안내: https://www.kopis.or.kr/por/cs/openapi/openApiInfo.do?menuId=MNU_00074
- 공공데이터포털 KOPIS 공연목록 API: https://www.data.go.kr/data/15097805/openapi.do
- Spring Boot 4.1.0 릴리스: https://spring.io/blog/2026/06/10/spring-boot-4/
- 토스페이먼츠 JavaScript SDK: https://docs.tosspayments.com/sdk/v2/js
- Caddy 리버스 프록시·정적 파일 서빙: https://caddyserver.com/docs/caddyfile/directives/reverse_proxy
- 토스페이먼츠 결제위젯 연동 가이드: https://docs.tosspayments.com/guides/v2/payment-widget/integration
- 토스페이먼츠 코어 API (승인·취소·조회): https://docs.tosspayments.com/reference
- 토스페이먼츠 멱등키: https://docs.tosspayments.com/reference/using-api/idempotency-key
- 토스페이먼츠 웹훅: https://docs.tosspayments.com/guides/v2/webhook
- 본 시스템의 좌석 배치와 회차는 데모용으로 생성한 데이터이며, 실제 공연장·예매처와 무관하다.